

Lab 11.1 - See sVirt Confine a VM

KVM Virtualisation on Oracle Linux - Day 2, Module 11

Overview

Module 11's guide explained that SELinux and sVirt have been enforcing isolation underneath every VM built in this course, without anything so far confirming it directly. This lab makes that enforcement visible and undeniable: you'll inspect the unique labels sVirt assigns to ol-lab-01's process and disk, then deliberately break the pairing by moving its disk to a path SELinux has no reason to trust - and watch the VM refuse to start as a direct result. You'll finish by fixing it the correct way, not by reaching for `setenforce 0`.

Learning Objective

By the end of this lab, you will be able to:

- Confirm SELinux is genuinely enforcing, not just installed
- Inspect a VM's unique `svirt_t` process label and its disk's matching `svirt_image_t` label
- Trigger a real SELinux denial by moving a VM's disk to an unlabelled path
- Diagnose the denial using the audit log, rather than guessing at the cause
- Fix the denial correctly with `semanage fcontext` and `restorecon`, and confirm the VM starts again

Step by Step Guide

Prerequisite: Confirm SELinux Is Enforcing

1. Connect to your lab host:

```
ssh opc@<your-lab-instance-ip>
```

2. Confirm SELinux is genuinely enforcing, not permissive or disabled:

```
getenforce
```

```
sestatus
```

Expected result: `getenforce` prints `Enforcing`. If it prints `Permissive` or `Disabled`, stop here and flag it - nothing in this lab will behave as described otherwise, and that alone is worth fixing before continuing.

3. Confirm ol-lab-01 exists and is running:

```
sudo virsh list --all
```

Step 1: Show Each VM's Unique svirt_t Label with ps -eZ

4. Find ol-lab-01's running QEMU process and its SELinux label:

```
ps -eZ | grep qemu
```

Expected result: A line showing a label resembling system_u:system_r:svirt_t:s0:c123,c456 - the svirt_t part is shared by every VM on the host, but the c123,c456 category pair is unique to ol-lab-01 specifically.

Note: If a second VM is running on this host (such as app-vm-01 from Lab 13.1), run this command again and compare the two category pairs directly - they should never match. That difference is the entire isolation mechanism, not a naming convention.

Step 2: Inspect a Disk Image's svirt_image_t Context with ls -Z

5. Check the SELinux label on ol-lab-01's disk file:

```
ls -Z /var/lib/libvirt/images/ol-lab-01.qcow2
```

Expected result: A label showing svirt_image_t and the exact same category pair noted in Step 1 - confirmation the process and its disk are labelled as a matching pair, which is what lets the kernel allow this specific process to access this specific file.

Step 3: Move a Disk to an Unlabelled Path - Watch the VM Fail to Start

Now break the pairing deliberately, by moving ol-lab-01's disk somewhere SELinux has no policy rule expecting VM storage to live.

6. Shut the VM down, confirming it has genuinely stopped:

```
sudo virsh shutdown ol-lab-01
```

```
watch -n 2 sudo virsh list --all
```

```
# Ctrl-C once state shows "shut off"
```

7. Create a new directory in a location SELinux does not associate with VM images, and move the disk there:

```
sudo mkdir -p /home/opc/svirt-test
```

```
sudo mv /var/lib/libvirt/images/ol-lab-01.qcow2 \  
/home/opc/svirt-test/ol-lab-01.qcow2
```

8. Check the disk's label at its new location:

```
ls -Z /home/opc/svirt-test/ol-lab-01.qcow2
```

Expected result: A label such as user_home_t or unlabeled_t - not svirt_image_t. The file kept whatever context its new parent directory carries; SELinux has no idea this is meant to be VM storage.

9. Update the VM's definition to point at the new path:

```
sudo virsh edit ol-lab-01
```

change the <source file='...'/> line under <disk> to:

```
# <source file='/home/opc/svirt-test/ol-lab-01.qcow2'/>
```

10. Attempt to start the VM:

```
sudo virsh start ol-lab-01
```

Expected result: The start fails with a permission error - typically mentioning Permission denied, even though the file's ordinary Linux ownership and mode would otherwise allow access. This is SELinux blocking it, not a filesystem permissions problem.

11. Confirm SELinux is genuinely the cause, rather than guessing:

```
sudo ausearch -m avc -ts recent
```

Expected result: A recent AVC denial listed, referencing the qemu process and the disk's path - direct evidence of what was blocked and why.

Step 4: Fix It the Right Way with restorecon and Confirm It Boots

The fix is not to disable SELinux - it's to tell its policy that this new path is expected to hold VM images, then apply that expectation to the file already there.

12. Register the new path with SELinux's file-context policy:

```
sudo semanage fcontext -a -t svirt_image_t \  
"/home/opc/svirt-test(/.*)?"
```

13. Apply that policy to the file that already exists there:

```
sudo restorecon -Rv /home/opc/svirt-test/
```

Expected result: restorecon reports it relabelled the file, showing the change from its old context to svirt_image_t.

14. Confirm the label is correct:

```
ls -Z /home/opc/svirt-test/ol-lab-01.qcow2
```

Expected result: The file now shows svirt_image_t.

15. Start the VM again:

```
sudo virsh start ol-lab-01
```

```
sudo virsh list --all
```

Expected result: ol-lab-01 starts successfully this time and is listed as running - the exact same disk, the exact same VM definition, the only change being that SELinux now has a policy rule covering this path.

Cleanup: Restore ol-lab-01 to Its Original Disk Location

Leave the environment as later labs expect to find it, rather than permanently pointing ol-lab-01 at a home directory.

16. Shut the VM down, move the disk back, and update the definition again:

```
sudo virsh shutdown ol-lab-01
```

```
watch -n 2 sudo virsh list --all
```

```
# Ctrl-C once state shows "shut off"
```

```
sudo mv /home/opc/svirt-test/ol-lab-01.qcow2 \  
/var/lib/libvirt/images/ol-lab-01.qcow2
```

```
sudo virsh edit ol-lab-01
```

```
# change the <source file='...'/> line back to:
```

```
# <source file='/var/lib/libvirt/images/ol-lab-01.qcow2'/>
```

```
sudo virsh start ol-lab-01
```

Expected result: ol-lab-01 running again from its original path - no restorecon needed this time, since /var/lib/libvirt/images already carries the correct svirt_image_t context by default.

Outcomes

By the end of this lab, you have seen sVirt's per-VM labelling directly rather than trusting it exists, and caused - then correctly fixed - a genuine SELinux denial rather than a simulated one. You now have direct, first-hand evidence of what Module 11's guide described: SELinux enforcement does not depend on the VM behaving correctly, it depends on the kernel's own policy, and the only correct way to extend that policy to a new location is semanage fcontext and restorecon - never disabling the protection itself.